

FRONT PAGE

Nombre Alumno / DNI	Oscar Ortega Rueda / 54291152C
Título del Programa	1º PHE CYBER / APPLIED COMPUTING / DATA SCIENCE
N.º Unidad y Título	UNIT 1 - Programming & coding
Año académico	2025-2026
Profesor de la unidad	Sergio Alonso Lopez
Título del Assignment	PHE APPLIED COMPUTING / CYBER SECURITY
Día de emisión	20/10/2025
Día de entrega	25/01/2026
Declaración del estudiante	Certifico que la presentación del assignment es completamente mi propio trabajo y entiendo completamente las consecuencias del plagio. Entiendo que hacer una declaración falsa es una forma de mala práctica. Fecha: 22/12/2025 Firma del alumno: 

Plagio

El plagio es una forma particular de hacer trampa. El plagio debe evitarse a toda costa y los alumnos que infrinjan las reglas, aunque sea inocentemente, pueden ser sancionados. Es su responsabilidad asegurarse de comprender las prácticas de referencia correctas. Como alumno de nivel universitario, se espera que utilice las referencias adecuadas en todo momento y mantenga notas cuidadosamente detalladas de todas sus fuentes de materiales para el material que ha utilizado en su trabajo, incluido cualquier material descargado de Internet. Consulte al profesor de la unidad correspondiente o al tutor del curso si necesita más consejos.

ÍNDICE

1. Introducción

- 1.1. Importancia de los lenguajes de programación
- 1.2. Rol de los paradigmas en el desarrollo de software

2. Tipos de Lenguajes de Programación

- 2.1. Lenguajes de bajo nivel
 - > Lenguaje máquina
 - > Ensamblador
 - > Usos más comunes
- 2.2. Lenguajes de medio nivel
 - > Características principales
 - > Ejemplos y aplicaciones
- 2.3. Lenguajes de alto nivel
 - > Características principales
 - > Ejemplos y aplicaciones
- 2.4. Lenguajes compilados e interpretados
 - > Definiciones y diferencias
 - > Ventajas y desventajas
 - > Selección según requerimientos de un proyecto

3. Paradigmas de Programación

- 3.1. Paradigma estructurado
 - > Principios fundamentales y conceptos clave
 - > Aplicaciones y ventajas
- 3.2. Paradigma orientado a objetos (POO)
 - > Principios fundamentales y conceptos clave
 - > Aplicaciones y ventajas
- 3.3. Paradigma dirigido por eventos
 - > Conceptos clave y funcionamiento
 - > Aplicaciones y ventajas
- 3.4. Otros paradigmas (funcional)

4. Tipos de Datos y Estructuras de Control

- 4.1. Tipos de datos y sus ejemplos
- 4.2. Tipos de estructuras de control, usos y ejemplos

5. Conclusión

6. Bibliografía y anexos

1. INTRODUCCIÓN

Escenario: Somos el dueño de la empresa E-volution, una empresa orientada a la migración de datos, desarrollo de software y transformación digital, tenemos la oportunidad de tener una reunión con el CEO de la empresa a la que vamos a prestar el servicio (MSMK) para desarrollar un diseño eficiente y mostrar un pequeño ejemplo del software o herramientas que vamos a implementar. Para ello hemos creado una presentación que se dividirá en 3 fases.

Fase 1: Debemos mostrar los diferentes paradigmas que existen en la programación y una conclusión que responderá las siguientes preguntas. ¿Qué paradigma es ideal en base a la finalidad del proyecto, sector en el que implementaremos el diseño y su estructura? ¿Por qué el paradigma escogido es el ideal para desarrollar el diseño?

Fase 2: Debido a que el lenguaje que se va a utilizar en el diseño es python, deberemos mostrar un pequeño ejemplo de las capacidades que tiene este lenguaje

Fase 3: Deberemos crear un pequeño prototipo de ejemplo utilizando el paradigma más eficiente.

Fase 1

El objetivo de esta entrega es: Profundizar en el conocimiento teórico sobre los lenguajes de programación y los paradigmas existentes. El informe deberá ofrecer un análisis detallado, reflejando una comprensión clara de; Los tipos de lenguajes, paradigmas de programación y los tipos de datos y expresiones (condicionales entre otros). Se deberá añadir una conclusión final que relacione los paradigmas de programación en relación al proyecto y seleccione el más eficiente.

1.1 IMPORTANCIA DE LOS LENGUAJES DE PROGRAMACIÓN

Definición de marco común: Los lenguajes de programación son un conjunto de reglas y **sintaxis** que permiten a los humanos dar instrucciones a una computadora para realizar tareas específicas.

Importancia: Facilitan la creación de software o nuevas tecnologías digitales y su desarrollo, resuelven problemas, mejoran la productividad de un equipo o del ser humano y su eficiencia de tiempo invertido en la realización de tareas específicas.

1.2 ROL DE LOS PARADIGMAS EN EL DESARROLLO DE SOFTWARE

Definición de marco común: Los paradigmas de programación son estilos, modelos o enfoques en los que se estructura, diseña y escribe un programa informático, es decir, la filosofía de diseño y enfoque del código de un lenguaje usado para desarrollar un software o tecnología digital.

Importancia: Los paradigmas de programación estructuran y organizan el código, haciéndolo más legible, mantenible y organizado. Son importantes ya que definen y guían la forma de programar y marcan unos estándares de programación generales, es decir, las '*normas*' estándares a la hora de programar en un lenguaje en específico. Dichas normas están pensadas para que el lenguaje sea funcional, lógico y apto, y el software que se diseñe sea eficiente, mantenible y seguro. El paradigma escogido en un proyecto impacta sobre la eficiencia, escalabilidad y mantenibilidad del sistema y de la corporación en la que se implanta.

2. TIPOS DE LENGUAJES DE PROGRAMACIÓN

Existen varios tipos de lenguajes de programación, este apartado se puede estructurar y clasificar de la siguiente forma; **lenguajes de bajo nivel, lenguajes de medio nivel y lenguajes de alto nivel.** Esta clasificación se basa en el grado de abstracción del lenguaje respecto al hardware y su facilidad de uso, es decir, cuánto se aleja el lenguaje respecto a la forma en la que la computadora procesa la información y que tan difícil es para el ser humano comprenderlo.

2.1 LENGUAJES DE BAJO NIVEL

Son aquellos que están cerca del código máquina y hardware, es decir, **la transformación que conlleva el código o sintaxis para que la información pueda ser comprendida por una computadora es mínima**, esto se define como abstracción mínima. **Se caracterizan por ser más complejos de comprender** para el ser humano ya que su lógica de funcionamiento es más compleja. Algunos de los **lenguajes de bajo nivel más importantes son el lenguaje máquina y el ensamblador**, dos lenguajes que son explicados a continuación.

LENGUAJE MÁQUINA

Definición de marco común: Aquel lenguaje que una computadora o máquina puede ejecutar directamente sin necesidad de traducción.

El lenguaje máquina se compone de código binario (ceros y unos). Los ceros y unos se empaquetan o ensamblan en bits que crean varias secuencias, cada secuencia define una operación y una dirección de memoria específica. Este lenguaje es fundamental ya que sus datos reúnen las instrucciones para que una máquina funcione correctamente y con una lógica o coherencia para el ser humano.

ENSAMBLADOR

Definición de marco común: El lenguaje ensamblador es aquel que proporciona instrucciones a la cpu de una computadora. Utiliza mnemónicos (Llaves) para representar o definir cada instrucción.

El lenguaje ensamblador se traduce a lenguaje máquina mediante un ensamblador (programa que convierte instrucciones legibles a instrucciones que puedan ser entendidas por el código máquina de un ordenador mediante la cpu). Una vez la información sea transformada a lenguaje de código máquina (ceros y unos), esta información, (ceros y unos) es empaquetada en bits. Posteriormente la CPU capta las secuencias (conjunto de bits) y ejecuta las órdenes.

USOS MÁS COMUNES

El lenguaje de bajo nivel es usado para: Programación y desarrollo de Sistemas Operativos y de sus núcleos o kernels (SO), controladores de dispositivos o hardware (drivers), optimización del rendimiento de softwares, programas o aplicaciones y programación y desarrollo de sistemas embebidos (softwares diseñados para controlar y gestionar dispositivos electrónicos).

2.2 LENGUAJES DE MEDIO NIVEL

A diferencia de los lenguajes de bajo nivel, los lenguajes de medio nivel ofrecen equilibrio entre control del sistema y facilidad de programación.

CARACTERÍSTICAS PRINCIPALES

Los lenguajes de medio nivel combinan características de los lenguajes de bajo nivel y los lenguajes de alto nivel. Están definidos en el punto intermedio de abstracción y su comprensión del código, sintaxis o instrucciones es más fácil. Además permiten un control directo sobre la memoria y el hardware, y son más fáciles de usar que los lenguajes de bajo nivel debido a que su estructura es más legible y sencilla para el ser humano. Los programadores que usan lenguajes de medio nivel deben comprender tanto la lógica del programa como ciertos aspectos del hardware, ya que estos lenguajes permiten interactuar directamente con memoria y registros. El código escrito en un lenguaje de medio nivel es posteriormente traducido por el compilador a código máquina o ensamblador, es decir, al código que la computadora ejecuta.

EJEMPLOS Y APLICACIONES

El lenguaje de medio nivel se diferencia con el de bajo nivel en que es más fácil de comprender para el ser humano y su cercanía al hardware no es tan extrema como el lenguaje de bajo nivel.

Estos son algunos de sus usos más importantes: Desarrollo de sistemas operativos (Una parte de los núcleos o kernels), **Controladores de hardware** (drivers), **sistemas embebidos**, **optimizaciones de rendimiento** (softwares de alto más óptimos) y **compiladores y herramientas de desarrollo** (muchos compiladores y herramientas de desarrollo incorporan lenguajes de medio nivel como C++ o C)

Algunos de los ejemplos de lenguajes de medio nivel son: **C** (Sistemas operativos, controladores, sistemas embebidos y optimización de sistemas operativos), **C++** (Videojuegos, simulaciones, sistemas embebidos y aplicaciones de alto rendimiento), **Ada** (Aplicaciones donde la fiabilidad es fundamental y la seguridad es crítica; sistemas aeroespaciales, militares, automotrices, control de tráfico y control ferroviario) y **Pascal** (Educación y enseñanza, prototipos de softwares... debido a su mayor facilidad de aprendizaje y programación).

2.3 LENGUAJES DE ALTO NIVEL

A diferencia de los lenguajes de bajo y medio nivel, los lenguajes de alto nivel se caracterizan por ser lenguajes con una **abstracción alta** (su sintaxis referente al código que se emplea es sencilla y entendible para el ser humano), **esto permite con facilidad entender su código de programación, y por ende, desarrollar más fácilmente los softwares**, programas o aplicaciones, esto se define en una mayor productividad y un amplio recurso de herramientas y recursos.

CARACTERÍSTICAS PRINCIPALES

Las características principales de los lenguajes de alto nivel son las siguientes: **sintaxis sencilla y legible** (son lenguajes más fáciles de entender para el ser humano), **abstracción del hardware** (no se necesitan manejar detalles técnicos como registros o direcciones de memoria), **portabilidad** (Si se precisan los requerimientos necesarios, un mismo programa puede ejecutarse en diferentes ordenadores o máquinas), **eficiencia en el desarrollo** (permite desarrollar más rápidamente programas), **amplia estructura de datos** (permiten usar listas, diccionarios, objetos y otras estructuras sin necesidad de implementarlos desde cero), **amplia variedad de herramientas y recursos** (incluye múltiples herramientas que facilitan el uso del desarrollo de un programa, trabajos en ciberseguridad...).

EJEMPLOS Y APLICACIONES

Podemos diferenciar los siguientes lenguajes y sus usos comunes:

Desarrollo webs (Python, PHP, JavaScript, Ruby), **Ciencia de datos matemáticos y científicos:** (Python, R, MATLAB y Julia), **Desarrollo de aplicaciones móviles** (Swift, Kotlin), **Programación de sistemas o aplicaciones complejas** (C, C++, java y C#).

2.4 LENGUAJES COMPILADOS E INTERPRETADOS

En este punto, respecto a los lenguajes compilados e interpretados, se tratarán temas como; definiciones y diferencias, ventajas y desventajas y selección de lenguajes en un proyecto.

DEFINICIONES Y DIFERENCIAS

Un lenguaje compilado es aquel que traduce el código del programa a código máquina antes de su ejecución y un lenguaje interpretado es aquel que traduce y ejecuta el código línea por línea en tiempo de ejecución.

La diferencia está en que el lenguaje compilado traduce el código del programa a código máquina antes de su ejecución y el lenguaje interpretado ejecuta el código línea por línea en el tiempo de ejecución.

VENTAJA Y DESVENTAJAS

Los lenguajes compilados ofrecen mayor velocidad de ejecución y detección de errores temprana, siendo ideales para aplicaciones de alto rendimiento, mientras que los interpretados destacan por su flexibilidad, portabilidad y facilidad de depuración lo que los hace ideales para desarrollo rápido y prototipos.

SELECCIÓN SEGÚN REQUERIMIENTOS DE UN PROYECTO

Los factores a considerar son los siguientes: **Rendimiento** (los compilados tienen mejor rendimiento), **tiempo de desarrollo** (los compilados tienden a ser más lentos), **portabilidad** (los compilados suelen ser menos portables) y **complejidad** (los compilados son recomendados para proyectos grandes o críticos y los interpretados para proyectos pequeños o medianos).

3. PARADIGMAS DE PROGRAMACIÓN

Como explicamos anteriormente los paradigmas de programación son estilos, modelos o enfoques en los que se estructura, diseña y escribe un programa informático, es decir, la filosofía de diseño y enfoque del código de un lenguaje usado para desarrollar un software o tecnología digital.

Entre la variedad de paradigmas de programación podemos diferenciar los siguientes:

3.1 PARADIGMA ESTRUCTURADO

El paradigma estructurado es el más antiguo, su principio es el de organizar el código en bloques y estructuras lógicas, con el objetivo de facilitar la lectura y comprensión del programa, evitar el uso excesivo de saltos (GOTO) y promover el uso de estructuras de control como condicionales (IF, ELSE), bucles (FOR, WHILE) o funciones.

Los principios fundamentales del paradigma estructurado son los siguientes:

1. **Secuencia:** el programa se ejecuta instrucción por instrucción siguiendo un orden lineal y lógico. Esto implica que cada acción se ejecute o ocurra después de la anterior.
2. **Selección o decisión:** permite que el programa decida entre dos o más caminos de ejecución. Es implementado mediante estructuras condicionales (if, else, then, switch...)
3. **Iteración o repetición:** consiste en ejecutar un conjunto de instrucciones hasta que se cumpla una condición. Es implementado mediante el uso de bucles (for, while, do while...)

A diferencia del paradigma funcional, en el paradigma estructurado, una estructura puede cambiar su valor o su estado a medida que el programa se ejecute.

Ejemplo:

`int x = 5` (Estado inicial es `x=5`)

`x = x + 2` (El estado posterior cambia a `x=7`)

El paradigma estructurado es utilizado principalmente en: Desarrollo de sistemas administrativos, programas pequeños y medianos, procesamiento de datos, automatización de tareas repetitivas y en la educación y la enseñanza.

El uso del paradigma estructurado incorpora las siguientes ventajas: Es fácil de entender y de aprender, su código es claro y ordenado, es fácil de mantener, favorece la reutilización del código, permite una mejor planificación, es aplicable a múltiples lenguajes.

3.2 PARADIGMA ORIENTADO A OBJETOS (POO)

El paradigma orientado a objetos (POO) organiza el código en unidades llamadas “objetos”. Combinando datos y funciones en un solo bloque y posibilitando representar de manera estructurada los elementos.

Los principios fundamentales del paradigma orientado a objetos se basan en:

1. Abstracción: permite representar elementos del mundo real mediante clases, centrándose en lo que es relevante para la aplicación en particular. La abstracción simplifica el diseño de software.

Ejemplo de abstracción: Una clase llamada Auto tiene métodos como arrancar(), frenar() sin necesidad de mostrarle al programa el funcionamiento del motor.

2. Encapsulación: Consiste en ocultar o encapsular los datos y detalles de un objeto dentro en una clase, ocultando así su complejidad y permitiendo el acceso a través de métodos definidos. El encapsulamiento evita la manipulación directa de datos y mejora la seguridad de estos siendo implementado con modificadores de acceso como por ejemplo private, public o protected.

Ejemplo de encapsulación: Un objeto llamado CuentaBancaria oculta el saldo y solo permite modificarlo mediante métodos como depositar() o retirar().

3. Herencia: Permite crear clases nuevas a partir de clases ya existentes, heredando sus atributos y métodos.

Ejemplo de herencia: Una clase llamada ‘Estudiante’ puede heredar de la clase ‘Persona’ sus atributos y métodos, y además añadir nuevos atributos como ‘carrera’ o ‘matrícula’.

4. Polimorfismo: Varios objetos pueden responder de diferentes formas a un mismo método.

Ejemplo de polimorfismo: El método ‘dibujar()’ puede comportarse de manera distinta en las clases Círculo, Cuadrado o Triángulo.

El paradigma orientado a objetos se aplica comúnmente en: Desarrollo de software a gran escala (sistemas empresariales, ERP, CRM, plataformas bancarias), desarrollo de aplicaciones gráficas (JavaFX, Qt, Tkinter), desarrollo web (Django en python, Ruby on Rails, Laravel en PHP o Spring en Java), videojuegos (personajes, enemigos, proyectiles, niveles o físicas del juego entre otros), sistemas del mundo real (tráfico, sistemas ecológicos, simulación médica o modelado 3D), bibliotecas o APIs (las APIs modernas implementan el paradigma orientado a objetos) y desarrollo móvil (lenguajes como Kotlin, Swift o Java usan el paradigma orientado a objetos)

El uso del paradigma orientado a objetos incorpora las siguientes ventajas: Reutilización del código (la herencia y abstracción permiten construir nuevas clases basadas en otras existentes),

modularidad (el software se divide en objetos y clases, facilitando el mantenimiento y su depuración), **mantenibilidad y escalabilidad** (permite actualizar o ampliar partes del sistema sin afectar a todo el programa), **seguridad de datos** (gracias al encapsulamiento los datos quedan protegidos y accesibles solo mediante métodos controlados), **polimorfismo** (permite crear componentes flexibles que pueden adoptar distintos comportamientos según el tipo de objeto) y **menor duplicación del código** (los principios de herencia y abstracción evitan tareas repetitivas y promueven el diseño limpio).

3.3 PARADIGMA DIRIGIDO POR EVENTOS

En la programación dirigida por eventos la estructura y ejecución de un programa están determinados por eventos, como por ejemplo interacciones del usuario, mensajes del sistema o cambios de estado.

Los fundamentos y principios clave del paradigma dirigido por eventos se basan en:

1. Se estructura mediante eventos: Un conjunto de instrucciones que forman sucesos que ocurren en un sistema como

Ejemplo de un evento: Un usuario registrado o una pulsación de una tecla en el teclado.

2. Productor (Publisher): Componente que genera y emite un evento.

Ejemplo del productor: Cuando un cliente realiza una compra, el productor, mediante el servicio o módulo 'Pedidos' emite un evento, como por ejemplo el evento PedidoCreado.

3. Consumidor (Subscriber): Componente que recibe el evento y reacciona a este.

Ejemplo consumidor: El servicio o módulo de 'Facturación' recibe el evento PedidoCreado y genera una factura. El consumidor sería, en este caso, el servicio 'Facturación'.

4. Broker de eventos: El broker de eventos recibe los eventos y los entrega a los consumidores que les interesen. Sin el broker de eventos los servicios tendrían que comunicarse directamente y dependería unos de otros. Esto significa que si un consumidor se bloquea (no emitir una factura), los demás servicios y consumidores relacionados también lo harían.

5. Modelo publicar-suscribir: Este modelo se basa en lo siguiente; el productor publica eventos y los consumidores se suscriben a los eventos que les interesen. Esto permite que muchos componentes reaccionen a un mismo evento.

Ejemplo del modelo publicar-suscribir: varios consumidores (Facturación, Inventario o Notificaciones) pueden reaccionar a un mismo evento (PedidoCreado)

6. Asincronía: El productor no espera a que el consumidor responda. Esto permite que los eventos se envíen y procesen sin bloqueos.

Ejemplo de asincronía: El servicio de 'Pedidos' no se detiene o espera a que el consumidor 'Facturación' cree la factura.

7. Inmutabilidad: Un evento emitido no es modificable, si algo cambia, se genera un nuevo evento.

Ejemplo de inmutabilidad: Se emite un evento llamado PedidoCreado y posteriormente el pedido se cancela, en este caso, se emitirá un nuevo evento como por ejemplo PedidoCancelado.

8. Procesamiento lógico de eventos: Los consumidores procesan un mismo evento varias veces sin causar errores.

Ejemplo de procesamiento lógico de eventos: Si llega dos veces un evento PedidoCreado al que solo corresponde un cliente, el consumidor 'Facturas' no se generará dos facturas del mismo cliente.

Resumen del funcionamiento: Ocurre un suceso → Se genera un nuevo evento → El evento se envía al Broker de eventos → Los componentes que lo necesitan reaccionan y ejecutan instrucciones, como por ejemplo la generación de una factura al recibir el evento PedidoCreado.

Algunos de los ejemplos en los que se aplica el paradigma dirigido por eventos son los siguientes: **Sistemas de comercio electrónico / e-commerce** (cada acción del usuario genera eventos como por ejemplo; PagoConfirmado, PedidoCreado, ClienteRegistrado, CarritoActualizado), **Procesamiento en tiempo real o streaming** (es usado en plataformas que consumen grandes volúmenes de datos como por ejemplo en detección de fraudes, Recomendaciones en tiempo real...), **Microservicios** (es útil para comunicar microservicios sin acoplarlos y permite que cada microservicio reaccione a eventos), **Plataformas de notificaciones y mensajería** (se usa para envíos de correos electrónicos, SMS o envío de Alertas como por ejemplo una notificación de una contraseña actualizada correctamente), **Finanzas y banca** (es usado para registrar transacciones en tiempo real, analizar fraudes, y sincronizar saldos), **Videojuegos y aplicaciones interactivas:** (es usado en movimiento de un jugador en un videojuego, notificaciones de un servidor y eventos del entorno o mundo entre otros ejemplos) y **Sistemas industriales y de IoT:** (es usado en sensores de temperatura, presión, movimiento y estado de una máquina industrial entre otros ejemplos).

3.4 OTROS PARADIGMAS IMPORTANTES (FUNCIONAL)

Existen otros paradigmas importantes como el Paradigma funcional entre otros: El paradigma funcional es un estilo de programación que se centra en calcular resultados mediante funciones puras sin modificar el estado, es decir, el valor de las variables y otras estructuras es inmutable.

Ejemplo:

let x = 5

let y = x + 2 → la variable x sigue siendo 5 y no cambia su estado o valor

Los ámbitos en los que se utiliza son los similares al estructurado pero con una diferencia clave, se usa en áreas donde la inmutabilidad, la previsibilidad y la facilidad para paralelizar tareas son importantes:

Algunos de sus usos se aplican en: Desarrollo de software concurrente y paralelo, ciencia de datos y análisis estadístico, inteligencia artificial y algoritmos, desarrollo de software seguro y predecible, lenguajes y frameworks modernos como JavaScript, Scala, Haskell o Clojure

4. TIPOS DE DATOS Y ESTRUCTURAS DE CONTROL

En este punto trataremos los siguientes aspectos: Los tipos de datos y sus ejemplos y los tipos de estructuras de control, sus ejemplos.

4.1 TIPOS DE DATOS, SUS USOS Y EJEMPLOS

Entre los distintos tipos de datos existen:

1. Los datos primitivos son los fundamentales y básicos de un lenguaje y se componen de los siguientes tipos: **Enteros** (1, -1), **decimales** (1.2, -3,4), **de carácter** ('A'), **booleanos** (Verdadero y Falso) y **cadenas de texto** ('Esto es una cadena de texto').

Los datos primitivos se usan para almacenar en memoria números, letras y textos, es decir, valores lógicos que un programa necesita para operar.

Ejemplo de datos primitivos guardados en una variable (en memoria): edad = 25 / precio = 99.99 / activo = True / nombre = "Ana"

2. Los datos compuestos son aquellos que se forman a partir de la combinación de varios datos: arrays o vectores, listas, tuplas y diccionarios.

Los datos compuestos se usan para almacenar colecciones de datos como listas de productos, registros de usuarios, coordenadas...

Ejemplo de un diccionario que guarda múltiples tipos de datos: usuario = {"nombre": "Ana", "edad": 25, "activo": True, "Descripción": "Abc123*_-"} }

4.2 TIPOS DE ESTRUCTURAS DE CONTROL

Las estructuras de control son un conjunto de reglas que determina un orden o secuencia en el que se ejecutan las instrucciones de un programa. Las podemos clasificarlas en:

1. **Estructuras de selección:** **if / else** (ejecutan bloques de código según instrucciones), **switch / match** (eligen una opción entre varias)
2. **Estructuras de repetición:** **for** (bucle repetitivo que realiza una acción determinada), **while** (bucle que se repite mientras una condición se cumpla o no se cumpla), **do... while** (bucle que se repite al menos una vez).
3. **Estructuras de salto:** **break** (cerrar o salir de un bucle), **continue** (salto hacia la siguiente interacción), **return** (devuelve un valor y normalmente finaliza una función).
4. **Otras estructuras menos comunes:** **try / catch / finally** (manejan excepciones), **async / await** (programación asíncrona), **yield** (genera valores por secuencia), **estructuras event handlers** (manejo de eventos), etc.

5. CONCLUSIÓN

Una vez entendamos el funcionamiento de cada paradigma de programación podemos analizar las ventajas y desventajas que tiene cada paradigma en relación al diseño del proyecto y el sector en el que opera la empresa. El diseño debe ser eficiente, funcional, escalable y seguro. El sector en el que opera la empresa MSMK en la que implementaremos el diseño es el sector de la educación. Si analizamos los requisitos, debemos encontrar un paradigma que resuelva objetos definidos, abstractos o físicos, como puede ser una matrícula de un alumno, el propio alumno, el profesor, una clase de estudio, etc. Además debemos tener en cuenta la relación entre cada objeto o entidad, ya que cada objeto responderá a múltiples sucesiones, eventos o reacciones, por ejemplo, si un alumno se matricula en varios cursos, se deberán crear tantas matrículas como cursos pero la repetición del mismo alumno para cada matrícula.

A continuación se definirán las ventajas de cada paradigma en relación al proyecto y se definirán ejemplos realistas para su entendimiento y comprensión.

¡IMPORTANTE!

Se debe de aportar la información siguiente: En los ejemplos aportados al final del punto actual, en relación al paradigma dirigido por eventos, podemos encontrar la explicación, muy importante, que defiende la diferencia entre el paradigma dirigido por eventos y el orientado a objetos y su relación con el proyecto, dicha explicación se caracteriza por contener la distinción de caracteres coloreados en rosa

Ventajas del uso de los distintos paradigmas en el proyecto:

Ventajas del paradigma estructurado en relación al proyecto: El paradigma estructurado tiene como ventaja su gran variedad de estructuras, variantes y complejidad en relación a los paradigmas orientado a objetos y dirigido por eventos. Es el paradigma predominante en proyectos complejos y amplios ya que su escalabilidad es mayor. El paradigma estructurado es perfectamente válido para la creación de este proyecto, no obstante, su uso es más común en proyectos que contienen más variantes o problemas a resolver y su estructura es más compleja. Dicho de otra forma, es un paradigma demasiado complejo para el proyecto y más escalable, lo que da como resultado una complejidad innecesaria, posibles errores comunes o innecesarios cuando se realizan cambios, una optimización en el código y seguridad de este y su estructura inferior respecto a otros paradigmas. Esta explicación se detalla con ejemplos más adelante, en los siguientes puntos de la conclusión.

Ventajas del paradigma orientado a objetos: La principal ventaja que tiene el paradigma orientado a objetos en relación al proyecto es la seguridad y encapsulación de los datos que adopta. El paradigma orientado a objetos es usado comúnmente cuando queremos representar objetos o definiciones de la vida real. En este caso, permite centrarse en el valor y reacción de cada objeto sin necesidad de predefinir cómo funciona cada objeto internamente, es decir, los objetos no varían, sino que adoptan una definición única para cada uno de ellos, y cuando se crean, no existe la necesidad de definir cómo funciona internamente el objeto, simplemente se define como reacciona ante diferentes eventos que suceden en la vida real, permitiendo una estructura de código, simple, funcional, segura y optimizada. Esta explicación se detalla en los siguientes puntos de la conclusión y acompañada de ejemplos.

Ventajas del paradigma dirigido por eventos: El paradigma dirigido por eventos tiene como principal ventaja la simplicidad y rapidez de ejecución, respecto a la relación con el proyecto, no es el mejor paradigma ya que no gestiona de forma eficiente las relaciones entre entidades. En el proyecto se necesitan varios atributos o, eventos en este caso, para una sola entidad, objeto en este caso. Por ejemplo, un alumno puede matricularse en varios cursos a la vez, en el paradigma dirigido por eventos deberían de suceder tantos eventos como el número de cursos que estudia el estudiante, a su vez con las materias dadas por los profesores, su asignación de clases físicas (habitáculo donde se dan las clases), etc. Esto es poco eficiente en relación a las condiciones en el código, ya que se generan demasiados eventos y, un evento puede interferir en otro, la optimización no es del todo eficiente por la generación masiva de códigos y, además la herencia se complica mucho ya que si queremos heredar un atributo o clase con un condicional, deberíamos unir la coherencia de cada objeto heredado con

todos los atributos que ha generado cada objeto, esta explicación puede ser difícil de entender, es por ello que se expone con ejemplos en los siguientes puntos de la conclusión.

Ejemplo de la creación de una persona y sus reacciones como proyecto de ejemplo y el funcionamiento de esta en los distintos tipos de paradigmas

Paradigma estructurado: Al crear una persona en el paradigma estructurado debemos de definir todas sus características; edad, longitud y color de pelo, color de ojos, masa muscular, grasa, masa ósea, etc. Respecto al proyecto de MSMK todas estas características serían irrelevantes ya que la relevancia está en como reacciona la persona a una situación, como por ejemplo, golpear una pelota, y no otras características irrelevantes. Al golpear una pelota características como color de pelo o vestimenta sería irrelevantes

Paradigma orientado a objetos: Al crear un personaje en el paradigma dirigido por eventos simplemente se define la persona y sus reacciones en diferentes situaciones, por ejemplo, si el jugador es dado de alta en un equipo se crea otro objeto llamado parte de alta, en esta situación el jugador ha entrado en un equipo y la reacción de esta situación es que se ha creado un parte de alta o ficha de alta.

Paradigma dirigido por eventos: Al crear la persona en el paradigma dirigido por eventos sería similar que en el paradigma orientado a objetos, por ejemplo, el jugador es fichado por un equipo (evento), este evento crea ficha de jugador. El problema persiste en que un estudiante puede tener varias matrículas y estar en varios cursos, lo que refleja cada evento único como la creación de varios objetos, en este paradigma esta situación es errónea, ya que para cada objeto debe suceder un evento, **si queremos crear dos matrículas de un solo alumno deberán de suceder dos eventos diferentes, lo que genera la representación duplicada del mismo alumno para cada evento, mientras que en el paradigma orientado a objetos, un mismo evento de sucesión puede crear varias matrículas para este alumno, de tal forma que no suceden 2 eventos para 2 matrículas sino 1 evento para 2 matrículas del mismo alumno.** El problema de usar este paradigma en el diseño del proyecto es la complejidad del programa en relación a las relaciones que tienen unos objetos con otros y su herencia, un ejemplo es el siguiente: **un estudiante puede matricularse en varios cursos, pero solo puede matricularse en un curso avanzado si antes estuvo matriculado en un curso básico**, como comentamos anteriormente deberían de suceder tantos eventos como matrículas, la relación es 1:1 siempre, entonces, tendríamos muchos eventos relacionados a un alumno, lo que dificultará la búsqueda de si el alumno estuvo matriculado en un curso básico, si queremos además registrar la fecha, se crearán tantos eventos como actualizaciones de fechas, y así sucesivamente con todos los objetos, sus atributos y eventos que suceden, ya tenemos 3 eventos relacionados con el mismo objeto, así sucesivamente.

Se debe de explicar que en el paradigma dirigido por eventos no es del todo correcto hablar de objetos, ya que no existen definiciones como tal cuando hablamos de este paradigma, no obstante, la definición de objeto, clases, atributos o métodos está simplificada y, es perfectamente adaptable con el paradigma dirigido por eventos. Dicho de una forma más lógica, podemos hablar de objetos, eventos, atributos, métodos en los paradigmas dirigido por eventos y orientado a objetos, pero debemos de entender que su definición es la misma pero su comportamiento no es el mismo en cada uno de los paradigmas. a continuación se explican las definiciones reales de estos conceptos en el paradigma dirigido por eventos.

Objeto:

En el paradigma dirigido por eventos, un objeto no es el eje central del diseño, sino un **elemento pasivo** sobre el que ocurren eventos. Un objeto puede representar a un alumno, un curso o una matrícula, pero **no encapsula toda la lógica de este**. Su función principal es servir como **referencia** para identificar sobre qué entidad ocurre un evento.

Atributo:

Los atributos en este paradigma no se mantienen necesariamente como parte permanente de un objeto. En muchos casos, los atributos son **datos asociados a un evento**, como el identificador del alumno, el curso o la fecha de la acción. Estos atributos describen el contexto del evento, pero no garantizan que el objeto conserve su estado de forma continua, ya que el estado puede reconstruirse a partir de la secuencia de eventos.

Evento:

Un evento es el **elemento principal del paradigma dirigido por eventos**. Representa un suceso concreto que ocurre en el sistema y provoca una reacción, como *alumno matriculado*, *fecha de matrícula actualizada* o *alumno dado de baja*. Cada evento contiene la información necesaria para ser procesado y ejecuta una acción mediante un manejador de eventos.

Clase:

Una **clase en el paradigma dirigido por eventos** se utiliza principalmente como una **estructura de apoyo**, no como el núcleo del sistema. Su función suele ser **definir eventos, agrupar datos relacionados o implementar manejadores de eventos**, pero no representar entidades del mundo real con estado y comportamiento propios.

Como conclusión resumida podemos definir que el paradigma estructurado es innecesario por su complejidad y el paradigma dirigido por eventos no cumple con la función y los requisitos propuestos respecto a las relaciones entre entidades. Mientras que el paradigma orientado a objetos cumple perfectamente con la complejidad básica necesaria y la variedad de distintas relaciones entre entidades o objetos.

5. BIBLIOGRAFÍA Y ANEXOS

AB-1 Lenguajes y paradigmas

Conceptos generales:

Lenovo (2025), *¿Qué son los lenguajes de programación y por qué son tan importantes en la tecnología?*, disponible en: [Enlace Lenovo](#).

Tipos de lenguajes y sus ventajas:

Grey. R. (2025), *¿Qué es el lenguaje de bajo nivel?*, ninjaOne, disponible en: [Enlace ninjaOne](#).

Alvarez. S. (2022), *Tipos de lenguajes de programación*, desarrolloweb, disponible en: [DesarrolloWeb](#)

Equipo Lenovo (2025), *¿Qué es un lenguaje de alto nivel?*, Lenovo, disponible en: [Enlace Lenovo](#).

Equipo Lenovo (2025), *¿Qué es un lenguaje de bajo nivel?*, Lenovo, disponible en: [Enlace Lenovo](#).

Equipo Lenovo (2025), *¿Qué es el lenguaje máquina?*, Lenovo, disponible en: [Enlace Lenovo](#).

Paradigmas:

Martinez Canelo M. (2025), *¿Qué son los paradigmas de programación?*, Profile, disponible en: [Enlace Profile](#).

Shaibu S. (2024), *¿Introducción a los paradigmas de programación?*, Datacamp, disponible en: [Enlace Datacamp](#).

Tipos de datos y estructuras de control:

Llamas L. (2025), *¿Qué son los tipos de datos*, luisllamas, disponible en: [Enlace luisllamas](#).

Sesmero M.P. et al. (2025), *Estructuras de control*, Universidad Carlos III de Madrid, ocw, disponible en: [Enlace ocw](#).

Maldonado R. (2025), *¿Qué son las estructuras de control en programación*, Keepcoding, disponible en: [Enlace Keepcoding](#).

Ortega Rueda Óscar, 2026

Fase 2

****El objetivo de este trabajo es el siguiente:**

1. Crear un algoritmo que calcule números pares e impares de una secuencia.
2. Pedir la ingesta de números separados por espacios
3. Manejo de errores y optimización del algoritmo

****El ejercicio se compondrá de dos archivos, archivo de main (Ejecutable) y archivo llamado numeros (archivo que contiene la lógica del programa), ambos archivos serán creados con la tecnología de python.**

Ejercicio completo

Main.py

```
from numeros import obtener_numeros, suma_numeros_pares, suma_numeros_impares

def main():

    # Obtener lista de números

    lista_numeros = obtener_numeros()

    print("Los números ingresados son:", lista_numeros)

    # Calcular suma de pares e impares

    suma_pares = suma_numeros_pares(lista_numeros)

    suma_impares = suma_numeros_impares(lista_numeros)

    print(f'La suma de los números pares es: {suma_pares}')

    print(f'La suma de los números impares es: {suma_impares}')
```

```
# Función que reemplaza name por main para poder unir los dos archivos python

if __name__ == "__main__":

    main()
```

****En el contenido del archivo de main.py se igualan el resultado que devuelve cada función a una variable para poder definir su resultado y escribirla en consola tantas veces como queramos sin necesidad de volver a ejecutar cada función de nuevo.**

```
if __name__ == "__main__":

    main()
```

****El anterior extracto de código es usado para poder importar el archivo main.py a otros archivos existentes.**

numeros.py

```
# Función que pide la ingesta de números como secuencia al usuario
def obtener_numeros():

    lista_numeros = [] # La secuencia que se pide a continuación se guardará en esta lista

    # Petición de la ingesta de números como secuencia
    secuencia_numeros_enteros = input('Introduce una secuencia de números enteros separados por espacios:\n')

    # TRATADO Y LIMPIEZA DE DATOS

    # Limpiar la entrada de espacios extra

    secuencia_numeros_enteros = secuencia_numeros_enteros.strip()

    # Bucle para intentar convertir a número los cada uno de los números o caracteres de la lista
    for caracter in secuencia_numeros_enteros.split():
        # Si se consigue convertir a número se añade a la lista
```

```

try:

    numero = int(caracter)

    lista_numeros.append(numero)

# Guardado de caracteres inválidos para la siguiente petición
except ValueError:

    print(f"'{caracter}' no es un número entero.")

# Petición para nueva ingesta de número válido
pregunta_cambio_numero = input(f"¿Deseas cambiarlo por otro número?
(sí/no): ").strip().lower()

# Si la respuesta es si pedir número y añadirlo a la lista
if pregunta_cambio_numero in ('sí', 'si'):

    pregunta_numero_nuevo = input('Introduce solamente un nuevo número:
').strip()

    try:

        numero_nuevo = int(pregunta_numero_nuevo)

        lista_numeros.append(numero_nuevo)

        print(f"'{caracter}' ha sido reemplazado por {numero_nuevo}.")

    except ValueError:

        print(f"El valor introducido ({pregunta_numero_nuevo}) no es un
número válido y será ignorado.")

# Si la respuesta es no comunicar al usuario que el valor será ignorado
elif pregunta_cambio_numero == 'no':

    print(f"El valor '{caracter}' será ignorado.")

else:

    print(f"Respuesta no válida. El valor '{caracter}' será ignorado.")

# Devolver la lista de números (secuencia limpia)

```

```

return lista_numeros

# Función de suma de pares de la secuencia

def suma_numeros_pares(lista_numeros):

    # Bucle, si el resto es 0, el número es par, entonces guardarlos en la lista de
    números pares
    numeros_pares = [num for num in lista_numeros if num % 2 == 0]

    # Sumar los valores de la lista de números pares y devolver el resultado
    return sum(numeros_pares)

# Función de suma de impares de la secuencia

def suma_numeros_impares(lista_numeros):

    # Bucle, si el resto es distinto a 0, el número es impar, entonces guardarlos en la
    lista de números impares
    numeros_impares = [num for num in lista_numeros if num % 2 != 0]

    # Sumar la lista de números impares y devolver el resultado
    return sum(numeros_impares)

```

****En el archivo numeros.py se realizan las siguientes secuencias**

1. Definición de la primera variable (petición de números)
2. Guardado de cada valor de la ingesta de la secuencia en una lista
3. Tratado de valores, se intenta convertir a número, sino se consigue se pide al usuario si desea cambiarlo por otro número, si lo desea se pide la ingesta de número válido y se añade a la lista si este es válido o correcto, si la ingesta del usuario no es válida el valor se ignora y no se añade, si el usuario no desea cambiar el valor de la lista inválido se le comunicará que el valor será ignorado. Si se consigue convertir a número cada valor de la secuencia que el usuario introdujo se añade cada valor a la lista. Por último la función devuelve el resultado, en este caso la variable que guarda la lista de números como secuencia
4. Se define la función de suma de pares. Se realiza un bucle y se recorre cada valor de la lista que devolvió la función anterior, posteriormente, se realiza una división de cada valor de la lista entre 2, si el resto es 0, el número es positivo y se añade a la lista de números pares. Por último se suma cada valor de la lista de números pares y se devuelve el resultado en una variable

5. Se define la función de suma de impares. Se realiza un bucle y se recorre cada valor de la lista que devolvió la primera función, posteriormente, se realiza una división de cada valor de la lista entre 2, si el resto es diferente a 0, el número es impar y se añade a la lista de números impares. Por último se suma cada valor de la lista de números impares y se devuelve el resultado en una variable

Ventajas de esta estructura

****¿Porqué se desarrolló esta estructura de código y no otra?**

Esta estructura tiene como ventaja la capacidad de detectar caracteres no válidos en la secuencia numérica, lo que da como resultado la modificación en cualquier aspecto de estos caracteres inválidos, es decir, se pueden ignorar, eliminar, convertir a número, etc...

El usuario tiene la opción de cambiar cada carácter inválido por un carácter o secuencia válidos antes de que se ejecute la operación hasta que la secuencia sea correcta.

Si escribo la secuencia 1 2 3e, puede que necesite que la secuencia correcta sea 1 2 y 3, antes de ejecutar cualquier otra función o operación.

Si escribo 1 2 3a 4a 5a y 6a el programa pedirá el cambio de 3a 4a 5a y 6a por separado para cambiarlos por otro numero y no pedirá el cambio de todos los caracteres inválidos a la vez.

Si escribo 1a 2a 1a 2a el programa pedirá el cambio de los 4 caracteres por separado sin contar que 1a y 1a es el mismo carácter. es decir se puede cambiar el primer 1a por un 5 y el segundo 1a por un 6.

Si en la secuencia 1, 2, 3a se ignora 3a, el resultado de la operación de la secuencia 1, 2 puede guardarse en una base de datos, lo que da como resultado mayor consumo de recursos y datos inválidos en una base de datos ya que el usuario debía de ingestar 1 2 3 como secuencia, pero se equivocó al pulsar las teclas lo que da como resultado el volver a tener que hacer la secuencia de nuevo o eliminar directamente los datos de la base de datos.

Fase 3

Como se comentó al principio del documento, en la fase 3 del proyecto se presentará una demo de proyecto, esta tendrá que ver con la creación de una clase con 'clases, estudiantes, sus atributos o datos y sus métodos. Se deberán crear dos clases, una llamada estudiantes y otra llamada cursos, cada estudiante tiene asignado una serie de atributos o datos del estudiante, como por ejemplo, nombre, edad, ciudad y hobby, promedio de notas, curso y si el estudiante es activo o histórico. Se debe crear una nueva clase que incorporará nuevos métodos para mostrar la versatilidad del programa.

El programa se compondrá de dos archivos, el archivo ejecutable llamado main.py y el archivo que contiene la lógica del programa llamado estudiante.py

Explicación del funcionamiento de cada uno de los archivos:

Main.py

```
# Importación de librerías necesarias

import re

from estudiante import Estudiante, Escuela

# Uso de restricciones y modificaciones para la limpieza de datos

def limpiar_nombre(nombre): # Definición de la función de limpieza de nombres

    return re.sub(r'^A-Za-záéíóúÁÉÍÓÜüÜñÑ ]', '', str(nombre).strip())
# El campo de nombre de estudiante solo puede contener letras, sin espacios y solo
podrá contener el formato string, todos los caracteres que no sean letras se
eliminarán

def limpiar_edad(edad): # Función limpieza del valor de edad

    numeros = re.sub(r'\D', '', str(edad)) # Solo se permiten números

    return int(numeros) if numeros else None # Convierte de formato string a número

def limpiar_texto(texto): # Función de limpieza del valor de hobby
```

```

    texto_limpio = re.sub(r'^A-Za-záéíóúÁÉÍÓÚüÜñÑ ', '', str(texto).strip()) #
solo se permiten letras y los valores se convierten a strings

    return texto_limpio if texto_limpio else "Campo sin rellenar" # En caso de que
el campo de hobby no este completo se qasignará por defecto el string "Campo sin
rellenar"

def limpiar_curso(curso): # Función impieza de los valores del campo de curso

    curso_limpio = re.sub(r'^A-Za-z0-9áéíóúÁÉÍÓÚüÜñÑ ', '', str(curso).strip()) #
Solo se premiten letras en formato string

    return curso_limpio

def obtener_nombre_valido(nombre_raw): # Definición de función nombre obligatorio

    while True:

        nombre_limpio = limpiar_nombre(nombre_raw)

        if nombre_limpio:

            return nombre_limpio

        nombre_raw = input("Por favor, el campo 'nombre' es obligatorio, rellénalo:
")

        # Preguntar al usuario que ingrese un nombre válido para el estudiante, ya
que este campo esta incompleto o es incorrecto

def obtener_edad_valida(edad_raw): # Definición de campo obligatorio

    while True:

        edad_limpia = limpiar_edad(edad_raw)

        if edad_limpia and edad_limpia > 0:

            return edad_limpia

```

```
edad_raw = input("Por favor, el campo 'edad' es obligatorio, rellénalo  
(número entero positivo): ")
```

```
# Preguntar al usuario que ingrese una edad válida para el estudiante, ya que este  
campo esta incompleto o es incorrecto
```

```
def obtener_curso_valido(curso_raw, estudiante): # Definición de campo obligatorio
```

```
    while True:
```

```
        curso_limpio = limpiar_curso(curso_raw)
```

```
        if curso_limpio:
```

```
            return curso_limpio
```

```
        curso_raw = input(f"La clase asignada al estudiante {estudiante} está  
incompleta, por favor, rellénela: ")
```

```
# Preguntar al usuario que ingrese un curso válida para el estudiante, ya  
que este campo esta incompleto o es incorrecto
```

```
def obtener_promedio_valido(promedio_raw): # Defnición de campo incompleto
```

```
    """Si es float válido devuelve float, sino devuelve 'Promedio sin asignar'"""
```

```
    try:
```

```
        return float(promedio_raw) # Transformación a float
```

```
    except:
```

```
        return "Promedio sin asignar"
```

```
# Si el campo es incompleto o incorrecto devuelve el string "Promedio sin  
asignar"
```

```
def obtener_activo_valido(activo_raw, estudiante): # Definición del valor de campo  
activo
```

```

while True:

    if isinstance(activo_raw, bool): # El campo será boolemano, activo
significa que el estudiante es un estudiante activo en el centro y false significa
que es hitórico

        return activo_raw

    elif str(activo_raw).strip().lower() == "true":

        return True

    elif str(activo_raw).strip().lower() == "false":

        return False

    activo_raw = input(f"El campo 'activo' del estudiante {estudiante} es
incorrecto o está incompleto, "

                        "por favor, introduzca solo los valores permitidos (True
/ False): ")

    # Debido a que es un campo obligatorio el programa
pedirá el ingreso de un campo válido

def main(): # Definición de datos de los estudiantes

    datos_estudiantes = [

        ("Rafa Gamero", "18", "Madrid", "programar", "Python 101", 7.3, True), #
Campos incompletos

        ("Ana López", "21", "Barcelona", "leer", "Matemáticas101", 8.7, True),

        ("Luis Pérez", "25", "Valencia", "correr", "Historia201", 7.5, False),

        ("María García", "19", "Sevilla", "dibujar", "Física101", 9.0, True),

        ("Carlos Ruiz", "30", "Bilbao", "cocinar", "Química101", 6.8, False),

        ("Laura Sánchez", "23", "Granada", "viajar", "Literatura101", 8.2, True),

        ("Jorge Martín", "28", "Zaragoza", "fotografía", "Arte101", 9.1, True),

```

```
        ("Elena Torres", "20", "Salamanca", "escribir", "Biología101", 7.9, True),  
  
        ("Pablo Díaz", "26", "Oviedo", "tocar la guitarra", "Música101", 8.5,  
True),  
  
        ("Lucía Romero", "22", "Córdoba", "bailar", "EducaciónFísica101", 9.3,  
True)  
  
    ]
```

```
# Definición de una lista donde se guardan cada uno de los atributos de cada  
estudiante
```

```
estudiantes = []
```

```
# Bucle para conseguir recorrer cada uno de los atributos guardados en la lista  
de estudiantes
```

```
for datos in datos_estudiantes:
```

```
    nombre = obtener_nombre_valido(datos[0])
```

```
    edad = obtener_edad_valida(datos[1])
```

```
    ciudad = limpiar_texto(datos[2] if len(datos) > 2 else "")
```

```
    hobby = limpiar_texto(datos[3] if len(datos) > 3 else "")
```

```
# Prueba de definición de mínimo de caracteres para cada campo, esto demuestra la  
versatilidad del paradigma orientado a objetos
```

```
    curso = obtener_curso_valido(datos[4] if len(datos) > 4 else "", nombre)
```

```
    promedio = obtener_promedio_valido(datos[5] if len(datos) > 5 else "")
```

```
    activo = obtener_activo_valido(datos[6] if len(datos) > 6 else "", nombre)
```

```
    estudiante = Estudiante(nombre, edad, ciudad, hobby, curso, promedio,  
activo)
```

```
    estudiantes.append(estudiante)
```

```
# Crear escuela y agregar estudiantes

escuela = Escuela("Escuela Python", "Madrid")

for est in estudiantes:

    escuela.agregar_estudiante(est)


# SALUDOS

print('\n' + 100*"-" + '\n' + "--> SALUDAR A TODOS LOS ESTUDIANTES" + '\n' +
100*"-", '\n')

for est in estudiantes:

    est.saludar()


# PRESENTACIÓN

print('\n' + 100*"-" + '\n' + "--> PRESENTACIÓN DE TODOS LOS ESTUDIANTES" +
'\n' + 100*"-", '\n')

for est in estudiantes:

    est.presentarse()


# Métodos nuevos pedidos en el AB3


# MOSTRAR ESTUDIANTES DE LA ESCUELA

print('\n' + 100*"-" + '\n' + "--> INFORMACIÓN COMPLETA DE TODOS LOS
ESTUDIANTES" + '\n' + 100*"-", '\n')

escuela.mostrar_estudiantes()
```

```
# MOSTRAR ESTUDIANTES ACTIVOS

print('\n' + 100*"-" + '\n' + "--> ESTUDIANTES ACTIVOS" + '\n' + 100*"-", '\n')

escuela.mostrar_estudiantes_activos()


if __name__ == "__main__":

    main()
```

****Con ayuda de los comentarios en el programa se puede entender perfectamente el funcionamiento de este, no obstante se demuestra una breve explicación**

****Pseudocódigo**

1. ****Presentación de funciones que devuelven los valores de cada campo, se demuestra que se pueden crear funciones para la ingesta de cada campo y funciones que manejen el campo introducido, así como su limpieza o tratado de datos, es decir, el valor de una función anterior puede ser modificado por otra función.**
2. ****Limpieza de datos, definición de caracteres mínimos y máximos, tratado de caracteres inválidos y petición de valores de campos obligatorios debido a, campos incompletos o incorrectos.**
3. ****Definición de datos de cada campo y guardado de estos en una lista.**
4. ****Bucle que recorre los campos de cada lista.**
5. ****Definición de variables y potencial del paradigma, se pueden crear bucles para, obligar al programa a actualizarse, actualizar la lista al volver a agregar los estudiantes nuevos y la creación de métodos como saludar o presentarse.**
6. ****Definición del valor de main, en este caso, **name** se sustituye por **main**, esto permite realizar pruebas o cohesionar el programa con cada archivo sin necesidad de crear varios nombre definidos, esto es muy usado en programas profesionales ya que separa el código estructural del funcional.**

Archivo de estudiante.py

```
class Estudiante:

    def __init__(self, nombre, edad, ciudad, hobby, curso="Sin asignar",
promedio=0.0, activo=True):
```

```
# Validaciones de datos obligatorias para el manejo y entendimiento de errores en el programa
```

```
if not isinstance(nombre, str) or nombre.strip() == "":  
  
    raise ValueError("El nombre debe ser un texto no vacío")
```

```
if not isinstance(edad, int) or edad <= 0:  
  
    raise ValueError("La edad debe ser un número entero positivo")
```

```
if not isinstance(ciudad, str) or ciudad.strip() == "":  
  
    ciudad = "Campo sin rellenar"
```

```
if not isinstance(hobby, str) or hobby.strip() == "":  
  
    hobby = "Campo sin rellenar"
```

```
# Si el atributo curso no es un string o esta vacío se definirá por defecto como "Sin asignar"
```

```
if not isinstance(curso, str) or curso.strip() == "":  
  
    curso = "Sin asignar"
```

```
try: # Calcular el promedio de notas, en caso de no poder calcularlo definir por defecto "0", en caso de no poder definirlo se definirá el string "Promedio sin asignar"
```

```
    promedio = float(promedio)
```

```
    if promedio < 0:
```

```
        promedio = 0.0
```

```
except:
```

```
    promedio = "Promedio sin asignar"
```

```
if not isinstance(activo, bool):
```

```
    activo = True
```

```
self.nombre = nombre

self.edad = edad

self.ciudad = ciudad

self.hobby = hobby

self.curso = curso

self.promedio = promedio

self.activo = activo


# Método saludar a estudiantes

def saludar(self):

    print(f"Hola {self.nombre}, ¡Bienvenido/a!")


# Método los estudiantes se presentan

def presentarse(self):

    print(

        f"Hola, soy {self.nombre}. "

        f"Tengo {self.edad} años, "

        f"vivo en {self.ciudad} "

        f"y me gusta {self.hobby}."

    )


# Método para mostrar toda la información del estudiante

def informacion_completa(self):
```

```
        print(f"{self.nombre}, Edad: {self.edad}, Ciudad: {self.ciudad}, Hobby: {self.hobby}, "
              f"Curso: {self.curso}, Promedio: {self.promedio}, Activo: {self.activo}")

# Definición de la clase escuela

class Escuela:

    def __init__(self, nombre, ciudad):

        self.nombre = nombre

        self.ciudad = ciudad

        self.estudiantes = []

    def agregar_estudiante(self, estudiante):

        if isinstance(estudiante, Estudiante):

            self.estudiantes.append(estudiante)

    def mostrar_estudiantes(self):

        for e in self.estudiantes:

            e.informacion_completa()

# Método que muestra únicamente los estudiantes que estan activos

    def mostrar_estudiantes_activos(self):

        for e in self.estudiantes:
```

```
if e.activo:  
  
    e.informacion_completa()
```

**En esta parte del código se definen los atributos de cada clase, tanto estudiantes como cursos. Cada atributo se iguala a una variable que contiene el prefijo de self. , posteriormente se asignan condicionales para manejo de errores y entendimiento de estos, y limpieza y manejo de valores incorrectos.

**Por último, una vez sean tratados los datos, se realiza la ejecución de los métodos y sus condiciones y funcionamiento al final del archivo.

**¿Por qué la limpieza de cada campo se realiza en el archivo main.py y el archivo estudiantes?

**En muchos casos esto se debe al flujo del programa, es decir, lo que queremos mostrar es su potencial, por ejemplo, si un valor está vacío o sin asignar podemos mostrar el Sting "Valor sin asignar", esto forma parte del main.py ya que este archivo construye la estructura del programa, definiendo los valores principales de cada campo y, en algunos casos, funciones importantes y esenciales. En otro caso, con el archivo de estudiantes podemos tratar los datos de cualquier forma, su limpieza, modificación etc. Sino es posible, main.py definirá cada valor por defecto para evitar errores. Es decir, es una técnica utilizada para conseguir estructuración del código sin intervenir en el orden de este, por así decirlo, hemos creado dos valores, un valor por defecto, en el archivo de main.py, por lo cual este valor "Sin asignar" será usado como última opción, posibilitando su modificación en el archivo estudiante.py